

assignment9

DSA Assignment #9: 图 (1/3)

Updated 2026-05-05 21:15 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M28046: 词梯

bfs, <http://cs101.openjudge.cn/practice/28046/>

思路:

本题分为两个部分，先是根据单词之间的差异程度建图，再在图上跑BFS求最短路径。因为单词差异为一个字母共有4种情况，可以划分成4类，每一类内单词两两之间可以建边；用字典bucket存储每个“类”对应的单词，然后就能快速建边。

代码:

```
from collections import defaultdict, deque
def main():
    n = int(input())
    edges = [[] for _ in range(n)]
    buckets = defaultdict(list)
```

```

word_rank = {}
words = []
for i in range(n):
    word = input().strip()
    word_rank[word] = i
    words.append(word)
    for j in range(4):
        buckets[word[:j] + '_' + word[j+1:]].append(i)
for bucket in buckets.values():
    length = len(bucket)
    for i in range(length - 1):
        for j in range(i + 1, length):
            edges[bucket[i]].append(bucket[j])
            edges[bucket[j]].append(bucket[i])

fool, sage = input().split()
s = word_rank[fool]
t = word_rank[sage]
dist = [0] * n
prev = [-1] * n
q = deque([s])
dist[s] = 1
while q:
    u = q.popleft()
    if u == t:
        break
    for v in edges[u]:
        if not dist[v]:
            dist[v] = dist[u] + 1
            prev[v] = u
            q.append(v)
if prev[t] == -1:
    print('NO')
    return
ans = []
while t != -1:
    ans.append(words[t])
    t = prev[t]
print(' '.join(ans[::-1]))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52419106提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
from collections import defaultdict, deque
def main():
    n = int(input())
    edges = [[] for _ in range(n)]
    buckets = defaultdict(list)
    word_rank = {}
    words = []
    for i in range(n):
        word = input().strip()
        word_rank[word] = i
        words.append(word)
        for j in range(4):
            buckets[word[:j] + '_' + word[j+1:]].append(i)
    for bucket in buckets.values():
        length = len(bucket)
        for i in range(length - 1):
            for j in range(i + 1, length):
                edges[bucket[i]].append(bucket[j])
                edges[bucket[j]].append(bucket[i])
```

基本信息

#: 52419106
题目: 28046
提交人: 25n2500011619(xq)
内存: 6444kB
时间: 43ms
语言: Python3
提交时间: 2026-04-28 18:26:04

M433.最小基因变化

bfs, <https://leetcode.cn/problems/minimum-genetic-mutation/>

思路:

题目要求将一个基因序列 A 变化至另一个基因序列 B，需要满足以下条件：

序列 A 与 序列 B 之间只有一个字符不同；

变化字符只能从 'A'，'C'，'G'，'T' 中进行选择；

变换后的序列 B 一定要在字符串数组 bank 中。

因为题目中的基因库 bank 的长度较小，可以预处理出 bank 中每个基因的合法变化，存在邻接表中，然后进行 BFS，计算起点到终点变换的最少次数（即最少边数）。

代码：

```
class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: List[str]) ->
int:
    def check(s1, s2):
        cnt = 0
        for i in range(8):
            if s1[i] != s2[i]:
                cnt += 1
            if cnt > 1:
                return False
        return cnt == 1

    if endGene not in bank:
```

```

        return -1
    if startGene == endGene:
        return 0

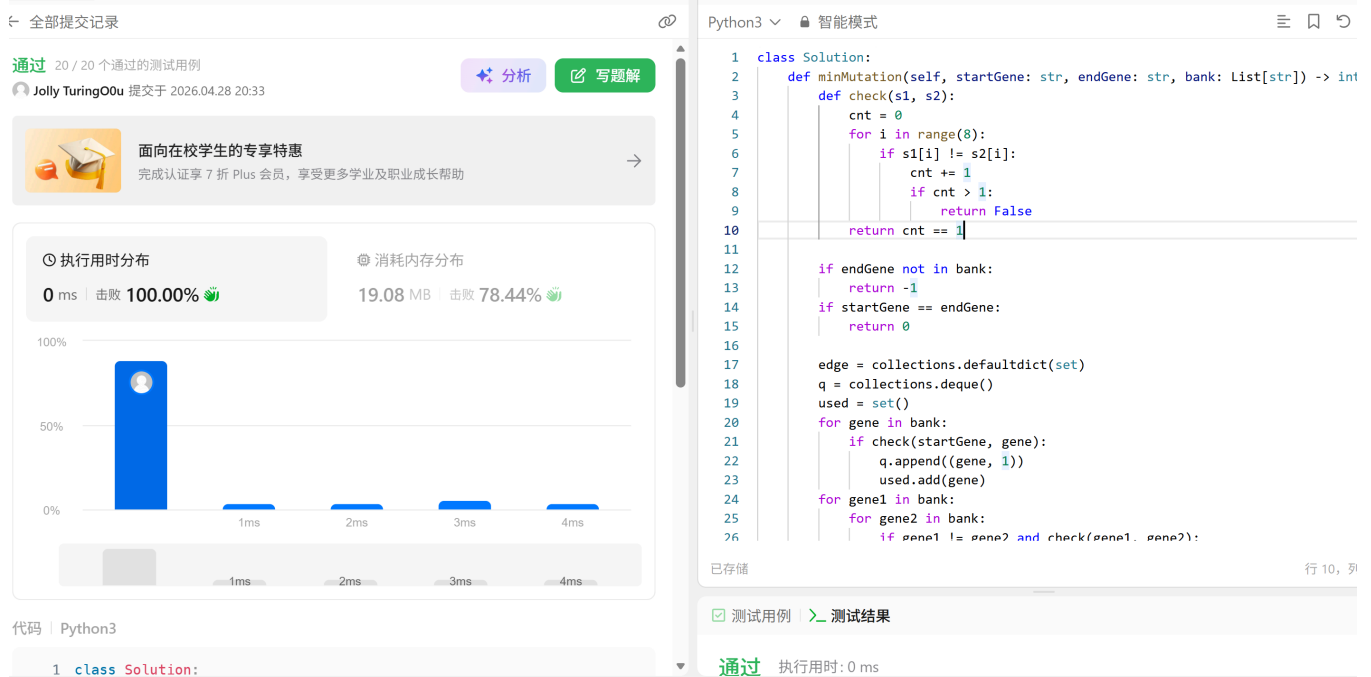
    edge = collections.defaultdict(set)
    q = collections.deque()
    used = set()
    for gene in bank:
        if check(startGene, gene):
            q.append((gene, 1))
            used.add(gene)
    for gene1 in bank:
        for gene2 in bank:
            if gene1 != gene2 and check(gene1, gene2):
                edge[gene1].add(gene2)

    while q:
        node, dist = q.popleft()
        if node == endGene:
            return dist
        for nxt in edge[node]:
            if nxt not in used:
                used.add(nxt)
                q.append((nxt, dist + 1))

    return -1

```

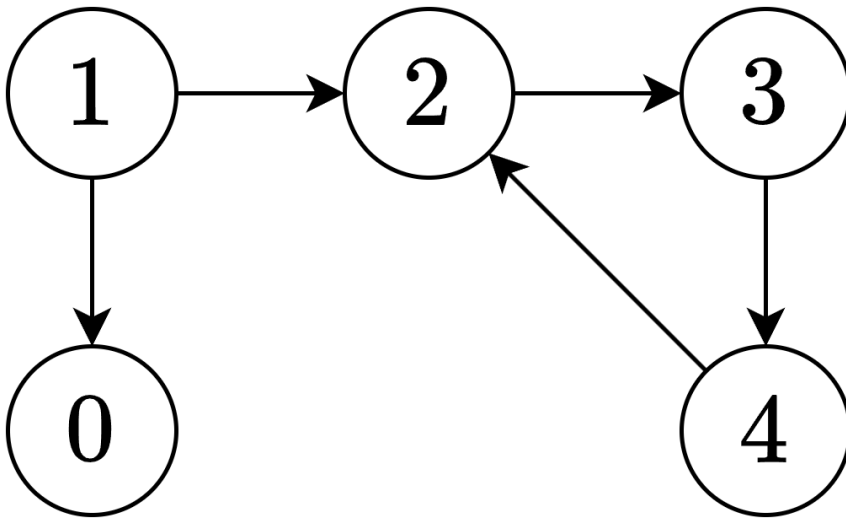
代码运行截图（至少包含有"Accepted"）



sy382: 有向图判环 中等

Karn, dfs, Floyd-Warshall, <https://sunnywhy.com/sfbj/10/3/382>

思路：



对于每个节点 x ，都定义三种颜色值（状态值）：

0：节点 x 尚未被访问到。

1：节点 x 正在访问中， $dfs(x)$ 尚未结束。

2：节点 x 已经完全访问完毕。注意这还说明从 x 出发无法找到环。所以当我们遇到状态值为 2 的节点 x 时，无需递归 x 。

⚠**误区**：不能只用两种状态表示节点「没有访问过」和「访问过」。例如上图，我们先 $dfs(0)$ ，再 $dfs(1)$ ，此时 1 的邻居 0 已经访问过，但这并不能表示此时就找到了环。

算法流程：

1. 建图：把每个 $prerequisites[i] = [a, b]$ 看成一条有向边 $b \rightarrow a$ ，构建一个有向图 g 。
2. 创建长为 n 的颜色数组 $colors$ ，所有元素值初始化成 0。
3. 遍历 $colors$ ，如果 $colors[i] = 0$ ，则调用递归函数 $dfs(i)$ 。
4. 执行 $dfs(x)$ ：
 - a. 首先标记 $colors[x] = 1$ ，表示节点 x 正在访问中。
 - b. 然后遍历 x 的邻居 y 。如果 $colors[y] = 1$ ，则找到环，返回 `true`。如果 $colors[y] = 0$ （没有访问过）且 $dfs(y)$ 返回了 `true`，那么 $dfs(x)$ 也返回 `true`。
 - c. 如果没有找到环，那么先标记 $colors[x] = 2$ ，表示 x 已经完全访问完毕，然后返回 `false`。
5. 如果 $dfs(i)$ 返回 `true`，那么找到了环，返回 `false`。
6. 如果遍历完所有节点也没有找到环，返回 `true`。

代码：

```
def check_loop():
    def dfs(u):
        colors[u] = 1 ## 正在访问
        for v in edge[u]:
            if colors[v] == 0 and dfs(v):
```

运行时长: 0 ms

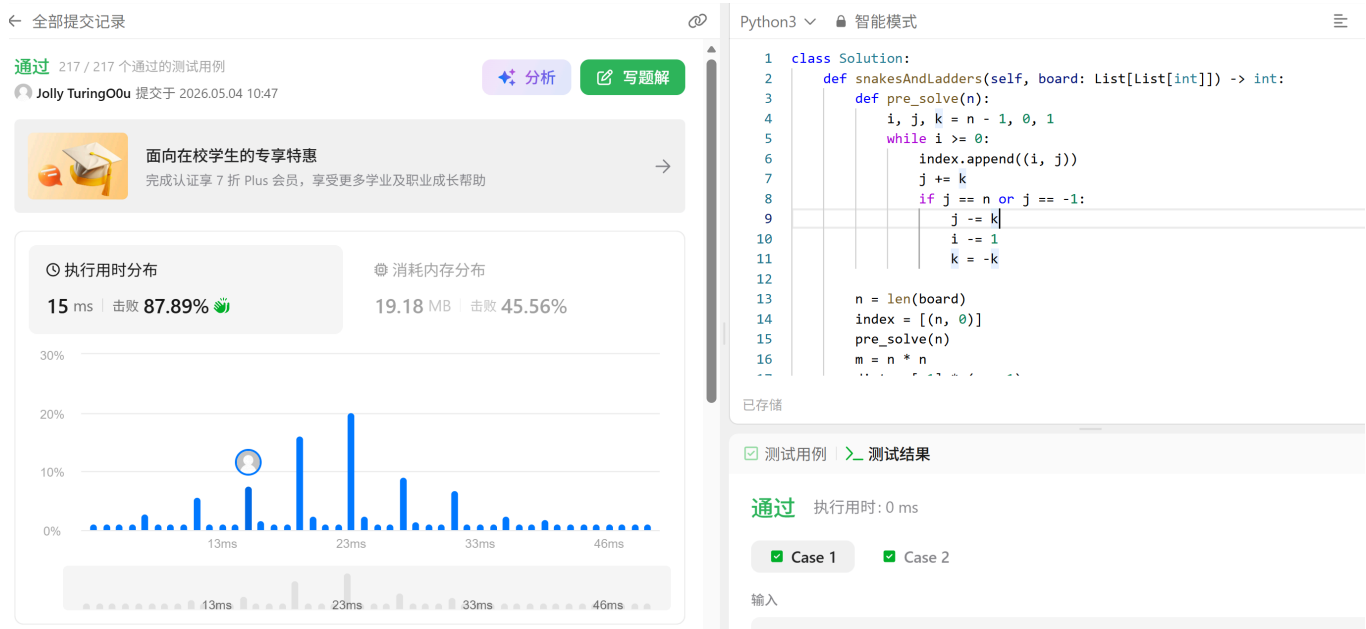
按照基本BFS算法实现，先预处理出每个编号对应的行列位置，然后进行广搜，如果 `board[r][c] != -1`，则跳到下一个对应位置，复杂度 $O(Dn^2)$ ，D为骰子的面个数。

代码：

```
class Solution:
    def snakesAndLadders(self, board: List[List[int]]) -> int:
        def pre_solve(n):
            i, j, k = n - 1, 0, 1
            while i >= 0:
                index.append((i, j))
                j += k
                if j == n or j == -1:
                    j -= k
                    i -= 1
                    k = -k

            n = len(board)
            index = [(n, 0)]
            pre_solve(n)
            m = n * n
            dist = [-1] * (m + 1)
            dist[1] = 0
            q = collections.deque([1])
            while q:
                u = q.popleft()
                if u == m: break
                disu = dist[u] + 1
                for nxt in range(u + 1, min(u + 6, m) + 1):
                    r, c = index[nxt]
                    if board[r][c] != -1:
                        nxt = board[r][c]
                        r, c = index[nxt]
                    if dist[nxt] == -1 or dist[nxt] > disu:
                        dist[nxt] = disu
                        q.append(nxt)
            return dist[m]
```

代码运行截图（至少包含有"Accepted"）



M28050: 骑士周游

dfs, <http://cs101.openjudge.cn/practice/28050/>

思路:

按照日字走法建边，使用启发式搜索(Warnsdorff 算法)，每次优先选择可访问节点数少的相邻节点访问。

代码

```
def dfs(step, u, limit):
    if step == limit:
        return True
    neighbors = ordered_by_avail(u)
    for v in neighbors:
        vis.add(v)
        flag = dfs(step + 1, v, limit)
        vis.remove(v)
        if flag: return True
    return False

def ordered_by_avail(u):
    res_list = []
    for v in edges[u]:
        if v not in vis:
            c = 0
            for w in edges[v]:
                if w not in vis:
```



```

        c += 1
        res_list.append((c, v))
    res_list.sort(key = lambda x: x[0])
    return [y[1] for y in res_list]

def gen_legal_moves(node_size):
    dr = [-1, -2, -2, -1, 1, 2, 2, 1]
    dc = [-2, -1, 1, 2, 2, 1, -1, -2]
    legal_moves = [[] for _ in range(node_size)]
    for u in range(node_size):
        c = u % n
        r = u // n
        for t in range(8):
            nr = r + dr[t]
            nc = c + dc[t]
            if 0 <= nr < n and 0 <= nc < n:
                legal_moves[u].append(pos_to_node_id(nr, nc))
    return legal_moves

def pos_to_node_id(r, c):
    return n * r + c

n = int(input())
edges = gen_legal_moves(n * n)
r0, c0 = map(int, input().split())
s0 = pos_to_node_id(r0, c0)
vis = {s0}
print('success' if dfs(1, s0, n * n) else 'fail')

```

(至少包含有"Accepted")

#52470906提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

def dfs(step, u, limit):
    if step == limit:
        return True
    neighbors = ordered_by_avail(u)
    for v in neighbors:
        vis.add(v)
        flag = dfs(step + 1, v, limit)
        vis.remove(v)
        if flag: return True
    return False

def ordered_by_avail(u):
    res_list = []
    for v in edges[u]:
        if v not in vis:
            c = 0
            for w in edges[v]:

```

基本信息

#: 52470906
 题目: 28050
 提交人: 25n2500011619(xq)
 内存: 3824kB
 时间: 22ms
 语言: Python3
 提交时间: 2026-05-05 17:39:12

T37.解数独

backtracking, hash table, <https://leetcode.cn/problems/sudoku-solver/>

思路：

使用三个哈希表分别存储行、列和宫格内的已填数字，同时使用二进制状态压缩节省空间。使用搜索时，同样使用启发式优化：优先填可填数字数量最少的格子；为了快速实现，可以使用最小堆存储元组 $(candidates, i, j)$ ，其中 $candidates$ 是 (i, j) 这个格子可填的数字个数，则堆顶就是 $candidates$ 最小的格子。

但是会遇到一个问题，在 (i, j) 格子填入数字后，同一行、列、宫格中的其他格子可填数字会减少，那么堆中的信息会不准确？对于本题的启发式搜索，即使过程中堆中信息可能会发生变化，但是因为可填数字个数只减不增，所以按照开始的顺序取仍是一种高效率的办法（可填数字个数是相对较小的），只需枚举完成之后撤销操作时更新实际的 $candidates$ 即可兼顾代码复杂度和效率。

代码

```
class Solution:
    def solveSudoku(self, board: List[List[str]]) -> None:
        """
        Do not return anything, modify board in-place instead.
        """
        tot_blank = 0
        h_flag = [0] * 9
        v_flag = [0] * 9
        b_flag = [0] * 9
        empty_pos = []
        for i, row in enumerate(board):
            for j, c in enumerate(row):
                if c == '.':
                    empty_pos.append((i, j))
                else:
                    x = int(c) - 1
                    h_flag[i] |= 1 << x
                    v_flag[j] |= 1 << x
                    b_flag[i // 3 * 3 + j // 3] |= 1 << x
        get_candidates = lambda i, j : 9 - (h_flag[i] | v_flag[j] | b_flag[i // 3 * 3 + j // 3]).bit_count()
        empty_heap = [(get_candidates(i, j), i, j) for i, j in empty_pos]
        heapq.heapify(empty_heap)

        def dfs():
            if not empty_heap:
```

```
    return True
```

```
_, i, j = heapq.heappop(empty_heap)
```

```
b = i // 3 * 3 + j // 3
```

```
candidates = 0
```

```
for x in range(9):
```

```
    y = 1 << x
```

```
    if h_flag[i] & y or v_flag[j] & y or b_flag[b] & y:
```

```
        continue
```

```
    candidates += 1
```

```
    board[i][j] = str(x + 1)
```

```
    h_flag[i] |= y
```

```
    v_flag[j] |= y
```

```
    b_flag[b] |= y
```

```
    if dfs(): return True
```

```
    board[i][j] = '.'
```

```
    h_flag[i] ^= y
```

```
    v_flag[j] ^= y
```

```
    b_flag[b] ^= y
```

```
    heapq.heappush(empty_heap, (candidates, i, j))
```

```
    return False
```

```
dfs()
```

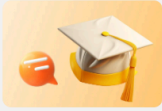
(至少包含有"Accepted")

通过 8 / 8 个通过的测试用例

Jolly Turing00u 提交于 2026.05.05 20:46

✦ 分析

✍ 写题解



面向在校学生的专享特惠

完成认证享 7 折 Plus 会员，享受更多学业及职业成长帮助

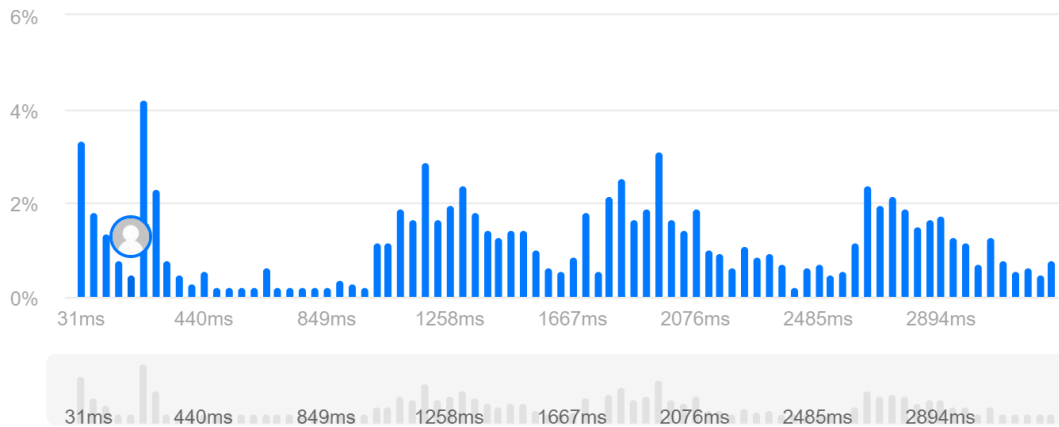


⌚ 执行用时分布

231 ms | 击败 91.95% 🏆

💾 消耗内存分布

19.60 MB | 击败 8.29%



2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

词梯和最小基因变化都是需要先建边再跑 *bfs*。区别是词梯的单词长度 C 较小而单词数目 n 较大，导致图非常稀疏，两两比较建边时间复杂度过高，需要利用等价类的想法将单词分组放入一个个 *bucket*，然后在各自的组内建边；而最小基因变化的 C 与 n 量级相当，可以用两两比较的方法建边。有向图判环是一个重要的问题，有向无环图(*DAG*)、拓扑排序等概念都与其相关，通过作业自己练习了 *DFS* 三色标记法和 *BFS* Kahn拓扑排序法，从不同角度分析这个问题，感觉收获很大。

骑士周游和填数独都使用了启发式搜索，每次优先选择可访问节点数少的节点访问，区别在于骑士周游的每个节点最多只有 8 条边相连，图比较稀疏，每次可以遍历子节点选择 *candidates* 最少的一个。而填数独类似全排列问题，只需确定每个位置填的先后顺序（通过小根堆来实现），然后依次枚举即可；同时还需要考察了 hash table 的使用，可以使用 set 或 二进制状态压缩。